

Day 14 - Capstone Project: Expense Tracker (in TypeScript)

Day 14 — Capstone Project: Expense Tracker (in TypeScript)

Objectives

Build a complete, small application in TypeScript, using nearly everything from the last 13 days, mostly unassisted:

- OOP (Days 8-9): classes for `Expense` and `ExpenseTracker`, private fields, inheritance if you find a natural place for it
- Async (Day 10): even though this project doesn't strictly need network calls, structure at least one operation (loading/saving) as `async`, since real-world file/data operations very often are
- TypeScript (Days 11-12): interfaces, a generic utility if you find a natural place for one, strict type annotations throughout
- Errors (Day 6): custom error classes, clean handling of bad input
- Testing (Day 12): a real Jest test suite, written by you, not given to you
- Structure (Day 13): proper `package.json`, `tsconfig.json`, project layout

This is intentionally the least hand-held day. You get a specification, not a scaffold. Expect to get stuck — that's the design, not a bug in the lesson.

The brief

Build a CLI expense tracker that:

- 1 **Adds expenses:** `node dist/index.js add 42.50 "Groceries" --category food`
- Each expense has: amount (number), description (string), category (string, defaults to `"general"`), and a timestamp (use `new Date()` when created)
- 1 **Lists expenses**, optionally filtered by category: `node dist/index.js list [--category food]`
- 2 **Shows a summary:** `node dist/index.js summary` — total spent, and a breakdown of total per category
- 3 **Exports to CSV:** `node dist/index.js export expenses.csv` (a simple hand-written CSV writer is fine — you don't need a package for this)
- 4 **Persists between runs** as JSON, exactly like Day 7, now inside a proper class rather than free functions, and written in TypeScript with real interfaces

Required design elements (this is the point of the exercise — don't skip these to save time)

- An `Expense` class (Days 8-9) with `toJSON()`/a static `fromJSON()` method for converting to/from a plain object suitable for `JSON.stringify/JSON.parse` (recall: `Date` objects don't survive a JSON round-trip automatically — you'll need to convert explicitly, similar in spirit to the Python track's Day 14 `to_dict/from_dict` pattern, if you've seen it).
- An `ExpenseTracker` class owning an array of `Expense` objects, exposing methods like `addExpense`, `filterByCategory`, `total`, `totalByCategory`.
- A custom error class, `InvalidExpenseError` extends `Error`, thrown when amount is negative or description is empty — caught at the CLI layer and shown as a clean message, never a raw, unhandled crash.
- Full TypeScript typing throughout — every function parameter and return type annotated, an `interface` for the plain-JSON shape of an expense, `strict` mode enabled in `tsconfig.json`.
- A real `tests/` directory with Jest tests covering: adding a valid expense, adding an invalid expense (throws), filtering by category, total calculations, and CSV export producing the right rows.
- Proper project structure (Day 13): `package.json` with build/test scripts, `tsconfig.json`, `src/`, `tests/`, `.gitignore`, and if you have git, real commits with meaningful messages as you go.

Suggested (not mandatory) file layout

```

day14-capstone/
  package.json
  tsconfig.json
  .gitignore
  src/
    models.ts      # Expense, InvalidExpenseError
    tracker.ts     # ExpenseTracker
    storage.ts     # load/save JSON, export CSV
    index.ts       # CLI entry point
  tests/
    models.test.ts
    tracker.test.ts

```

How to approach 2+ hours of mostly-unassisted work without stalling out

- 1 **Write the `Expense` class first, in isolation**, and test it manually (a small scratch `.ts` file you compile and run, or the Node REPL after compiling) before touching the CLI or storage at all.
- 2 **Build `ExpenseTracker` next**, still testing manually — add a few expenses, call `.total()`, before wiring up JSON persistence.
- 3 **Add JSON save/load**, confirm round-tripping works, including the `Date` conversion problem mentioned above.
- 4 **Only then** wire up the CLI (`process.argv` parsing, exactly like Day 7).
- 5 **Write tests as you go**, not all at the end.
- 6 If you get stuck for more than ~15 minutes on one specific thing, that's a reasonable point to peek at the relevant part of `solution/` for just that piece — not the whole file.

A hint on the Date-through-JSON problem, since it's genuinely new today

`JSON.stringify` automatically converts a `Date` object into a string (via its `toISOString()` method) — but `JSON.parse` has no way of knowing that string was originally a `Date`, and hands you back a plain string instead. To persist and correctly restore an `Expense` with its timestamp, you need explicit conversion methods:

```
interface ExpenseData {
  amount: number;
  description: string;
  category: string;
  timestamp: string; // stored as an ISO string, not a Date
}

class Expense {
  // ... constructor, etc ...

  toJSON(): ExpenseData {
    return {
      amount: this.amount,
      description: this.description,
      category: this.category,
      timestamp: this.timestamp.toISOString(),
    };
  }

  static fromJSON(data: ExpenseData): Expense {
    return new Expense(data.amount, data.description, data.category, new Date(data.timestamp));
  }
}
```

Definition of done

- All 5 CLI commands work end to end, tested manually.
- `npm test` passes with no failures, and covers the required cases listed above.
- No raw, unhandled crashes reach the user for the invalid-input cases you're asked to handle.
- `npm tsc` compiles with no type errors, with `strict: true` enabled.

Reference solution

`solution/` has a complete working implementation. Give yourself a genuine attempt — ideally the full 2 hours — before opening it. The value of this whole two weeks culminates here: if you can build this mostly on your own, in TypeScript, you're at the level this track promised.